

Organizzazione Fisica

Assunzioni Generali

Il **COSTO** corrisponde al **NUMERO DI ACCESSI al disco**, dove per accesso intendiamo “trasferimento di un blocco da memoria secondaria a memoria principale o viceversa”.

Un **BLOCCO** deve essere allocato per intero. NON è possibile avere dei record metà su un blocco e metà su altro.

(Es. se in un blocco entrerebbero 7.3 record ce ne mettiamo solo 7, e per i restanti 0.3 allochiamo un nuovo blocco)

Ogni **FILE** contiene **UN'UNICA TABELLA**. Cioè, ad ogni relazione corrisponde un file di record.

Ad ogni **ATTRIBUTO** corrisponde un **CAMPO**.

Ad ogni **TUPLA** corrisponde un **RECORD**.

Record

In un record oltre ai campi che corrispondono agli attributi ci possono essere altri campi che contengono informazioni sul record stesso o puntatori ad altri record/blocchi. All'inizio di un record alcuni byte possono essere utilizzati per:

- **Specificare il tipo del record** (è necessario quando in uno stesso blocco sono memorizzati record appartenenti a più file)
- **Specificare la lunghezza del record** (se il record ha campi a lunghezza variabile)
- **Contenere un bit di “cancellazione” o un bit di “usato/non usato”** (indica se il blocco è libero o meno)

Record a Lunghezza Fissa

Se tutti i campi del record hanno lunghezza fissa, l'inizio di ciascun campo contiene un offset, che indica il numero di byte dall'inizio del record.

Esempio: funziona sostanzialmente come l'indice di un Array. Se voglio prendere il campo X ed l'offset è 20, X si troverà in posizione “InizioFile+20”

Record a Lunghezza Variabile

Se il record contiene campi a lunghezza variabile:

- All'inizio di ogni campo si specifica la **lunghezza del campo** in numero di byte

OPPURE

- All'inizio del record ci sono gli **offset dei campi a lunghezza variabile** (tutti i campi a lunghezza fissa precedono quelli a lunghezza variabile)

Puntatori

Un puntatore ad un record/blocco è un dato che permette di accedere rapidamente al record/blocco. Esso può contenere:

- **L'indirizzo di inizio record/blocco** sul disco

OPPURE

- **Nel caso di un record, una coppia (b,k)** dove b è l'indirizzo del blocco e k è il valore della chiave

NOTA: Nel secondo caso è possibile spostare il record all'interno del blocco senza modificare il puntatore, nel primo no altrimenti potremmo avere dei dangling pointer (puntatori pendenti)

Blocchi

In un blocco ci possono essere: informazioni sul blocco stesso e/o puntatori ad altri blocchi.

Record a Lunghezza Fissa

Se un blocco contiene record di lunghezza fissa:

- **il blocco è suddiviso in sottoblocchi di lunghezza fissa ciascuna delle quali può contenere un record**
- **i bit “usato/non usato” sono raccolti in uno o più byte all’inizio del blocco.** (Altrimenti, se il bit “usato/non usato” è in ciascun record ciò può richiedere la scansione di tutto il blocco)

Record a Lunghezza Variabile

Se un blocco contiene record di lunghezza variabile:

- **Si pone in ogni record un campo che ne specifica la lunghezza**
- **Si pone all’inizio del blocco una directory contenente gli offset ai record** nel blocco

Directory di un Blocco

La directory può essere preceduta da un campo che specifica quanti sono i puntatori nella directory

OPPURE

Può essere una lista di puntatori (la fine della lista è specificata da uno 0)

OPPURE

Ha dimensione fissa e contiene il valore 0 negli spazi che non contengono puntatori

Operazioni sulla BD

Un’operazione sulla base di dati consiste di:

- **Ricerca**
- **Inserimento** (implica ricerca se vogliamo evitare duplicati)
- **Cancellazione** (implica ricerca)
- **Modifica** (implica ricerca)

È facile notare che la ricerca è alla base di tutte le altre operazioni. Pertanto sarà fondamentale scegliere un’organizzazione adeguata alle nostre esigenze, così da migliorare le prestazioni del DBMS, tenendo conto di quali operazioni saranno effettuate più frequentemente.

File Heap

È in realtà una **NON ORGANIZZAZIONE dei dati**. Infatti prevede di collocare i record nei file in base al loro ordine di arrivo. Questo fatto di **inserire i record sempre alla fine**, senza seguire nessun ordinamento, ci porta chiaramente ad avere le prestazioni peggiori di ricerca, ma inserimento molto veloce (se non abbiamo duplicati... altrimenti bisogna comunque effettuare ricerca per capire se quel record è già presente).

Ricerca

Se si vuole ricercare un record occorre scandire tutto il file, iniziando dal primo record fino ad incontrare il record desiderato. Questo comporta chiaramente che il costo della ricerca varia in base a dove si trova il record:

se il record che si cerca si trova nell'*i*-esimo blocco occorre effettuare i accessi in lettura

Pertanto ha senso valutare il **costo medio di ricerca**. Per trovare il costo medio sarà necessario sommare i costi necessari per trovare il singolo record e dividere il tutto per il numero di record totali. Supponiamo quindi:

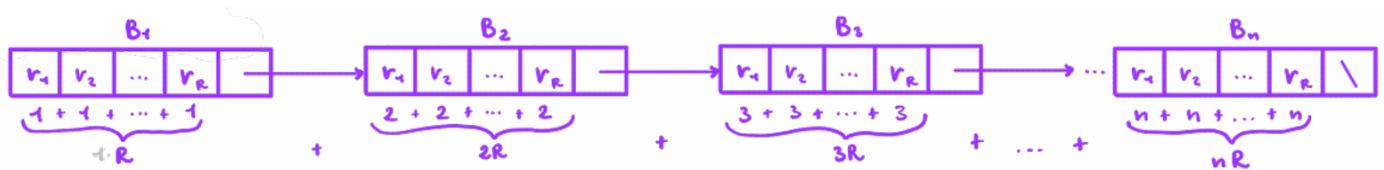
N: numero di record

R: numero di record che possono essere memorizzati in un blocco

Se ne deduce chiaramente che:

$n = N/R =$ **numero di blocchi** necessari per memorizzare i record

Avremo quindi che per accedere ai singoli record:



Per accedere ai primi R record
basta caricare il primo blocco

+ Per accedere ai successivi record bisogna
caricare il primo e il secondo blocco

+ ... + E così via... Bisogna sempre caricare tutti i
blocchi prima

E quindi la media è data da:

$$\text{COSTO_MEDIO} = \frac{R \times 1 + R \times 2 + \dots + R \times i + \dots + R \times n}{N} = \frac{R \times (1 + 2 + \dots + i + \dots + n)}{N} = \frac{R}{N} \times \frac{n(n+1)}{2} = \frac{1}{n} \times \frac{n(n+1)}{2} \approx \frac{n}{2}$$

$n = N/R \Leftrightarrow 1/n = R/N$

ATTENZIONE! Se cerchiamo **TUTTI** quelli che soddisfano una determinata chiave (es. tutti gli studenti di Roma) sarà comunque necessario scorrere **TUTTI** i blocchi, poiché non possiamo fermarci al primo che incontriamo $\Rightarrow$ **COSTO = n**

Inserimento (con duplicati)

Ci è sufficiente caricare in memoria principale (operazione di lettura) l'ultimo blocco.

Inseriamo il nuovo record e poi ricarichiamo in memoria secondaria (operazione di scrittura) il blocco aggiornato

$$\text{COSTO} = 1 + 1 = 2$$

Inserimento (senza duplicati)

Se **NON** ammettiamo duplicati, prima di inserire il record come ultimo dobbiamo accertarci che tale record non sia già presente nel DB. Quindi bisogna prima effettuare una ricerca.

$$\text{COSTO} = \text{ricerca} + \text{inserimento} = n/2 + 2$$

Modifica

Bisogna effettuare prima la ricerca, per trovare il record che vogliamo modificare, ed un accesso in scrittura per rimettere in memoria secondaria il blocco modificato.

$$\text{COSTO} = \text{Ricerca} + 1 = n/2 + 1$$

ATTENZIONE! Il costo di ricerca già comprende il costo di caricamento del blocco per la lettura.

Cancellazione

Quando effettuiamo una cancellazione di un record lasceremmo un buco nel blocco da cui è stato rimosso. Per colmare questo buco dovremmo compattare tutti quanti i record... chiaramente troppo costoso. Dunque quello che si fa è prendere l'ultimo record dell'ultimo blocco e lo si inserisce per colmare il buco. Questo ci permette di risparmiare sul costo di ricerca poiché supponiamo di avere un puntatore diretto all'ultimo blocco del file. Inoltre i record negli heap non sono ordinati, quindi li metto come voglio.

	matr	cognome	...
blocco 1	010	Rossi	...
	005	Verdi	...
	...	...	...
	031	Bianchi	...
blocco 2	048	Bellini	...
	123	Stella	...
	009	Carini	...
	...	...	...
...	...	...	...
blocco n	...	...	...

Costo ricerca

già comprende il costo per la lettura del blocco contenente i record cercati.

+

1 accesso in lettura

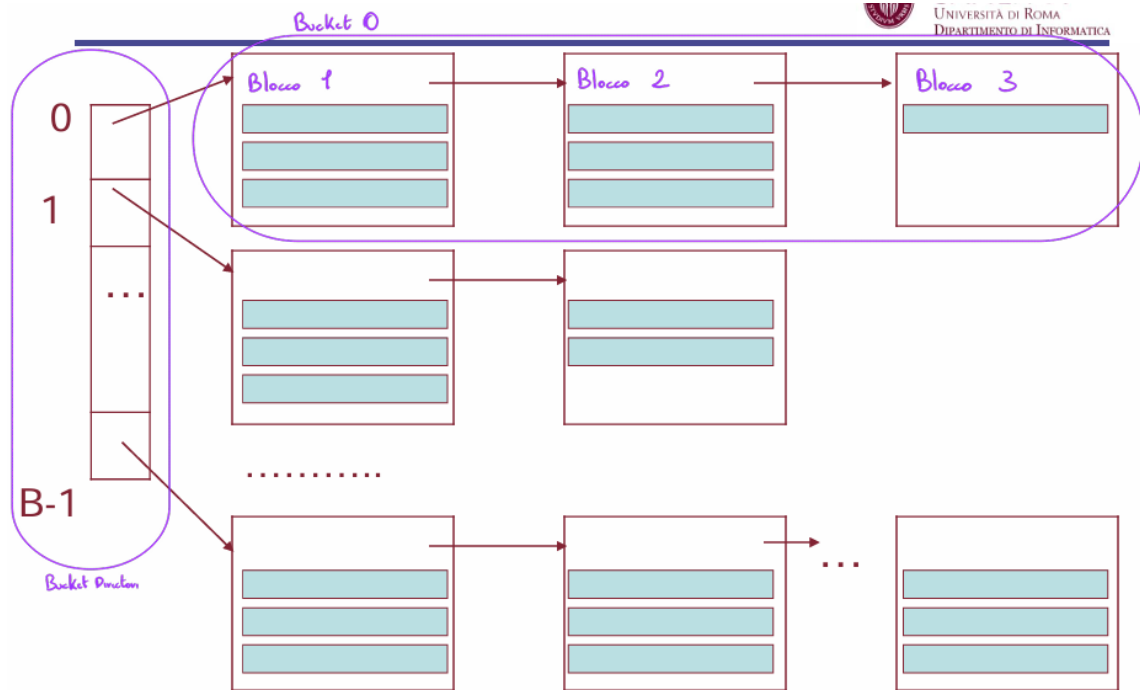
(per leggere l'ultimo blocco)

+

2 accessi in scrittura

(per riscrivere in memoria secondaria il blocco modificato e l'ultimo blocco)

File Hash



Buckets

Un File Hash è suddiviso in bucket numerati da 0 a B-1.

Ciascun bucket è costituito da uno o più blocchi, collegati mediante puntatori, che funziona come un heap.
(cioè i record nei blocchi non sono ordinati ma inseriti sempre alla fine)

Bucket Directory

L'accesso ai bucket avviene attraverso la bucket directory che contiene, per ogni bucket, un puntatore al primo blocco del bucket (*bucket header*).

Funzione hash

Il bucket in cui deve trovarsi un record con chiave v è calcolato mediante un'apposita funzione, detta funzione di hash.

Una funzione hash per essere "buona" deve ripartire uniformemente i record nei bucket, cioè al variare del valore della chiave deve assumere con la "stessa" probabilità uno dei valori compresi tra 0 e B-1. In genere, una funzione hash trasforma la chiave in un intero, divide questo intero per B, e fornisce il resto della divisione come numero del bucket in cui deve trovarsi il record con quel valore della chiave.

ATTENZIONE! La chiave di HASH può essere diversa dalla chiave relazionale.

Operazioni

Una qualsiasi operazione (ricerca, inserimento, cancellazione, modifica) su un file hash richiede:

$$\text{COSTO} = 0 + \text{Costo Operazione nell'Heap}$$

La valutazione di della funzione di hash per individuare il bucket
(che ha costo 0 poiché non richiede di caricare alcun blocco)

L'esecuzione dell'operazione sul bucket
(che è organizzato come un heap)

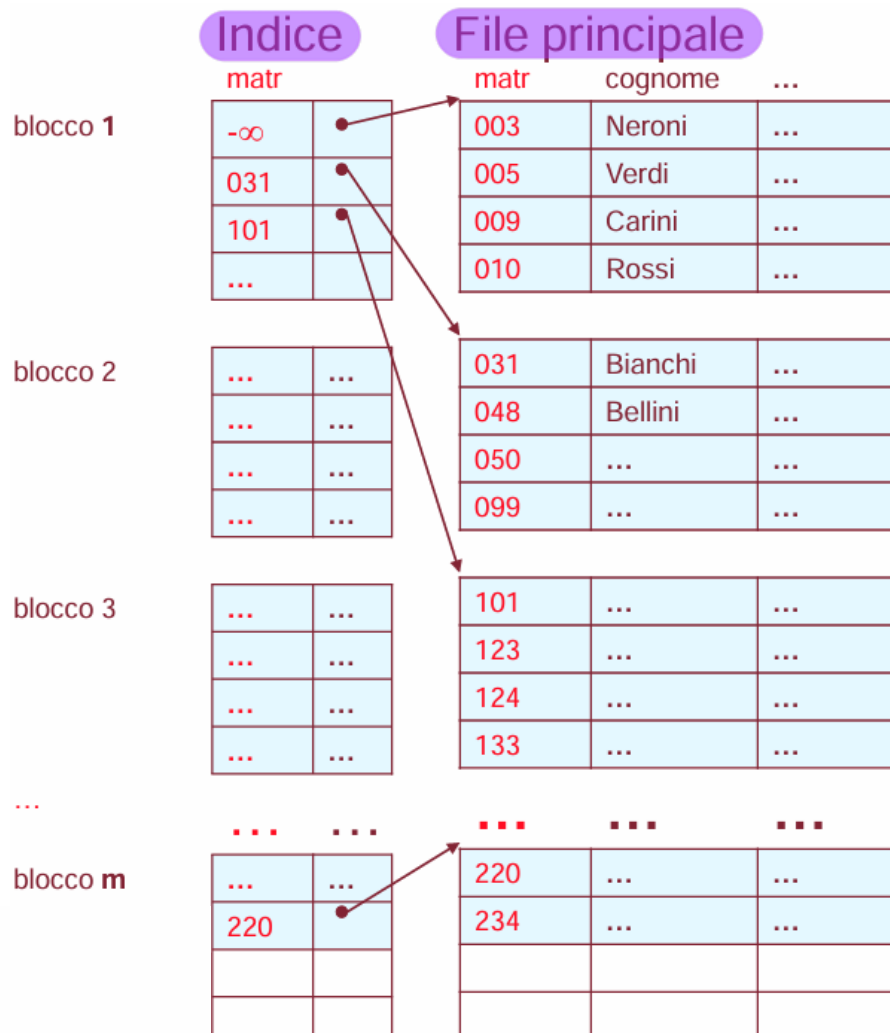
Poiché l'inserimento di un record viene effettuato sull'ultimo blocco del bucket, è opportuno che la bucket directory contenga anche, per ogni bucket, un puntatore all'ultimo record del bucket.

File con Indice

Quando le chiavi ammettono un ordinamento significativo per l'applicazione, è più conveniente utilizzare un'organizzazione fisica dei dati che ne tenga conto.

Interi e stringhe ammettono i consueti ordinamenti (lessicografico per le stringhe). Per campi multipli, si ordina sul primo campo, poi sul secondo e così via

ISAM



Il File ISAM - Indexed Sequential Access Method, viene ordinato in base al valore della chiave di ricerca. Per questo, anche negli esercizi, spesso viene richiesto di lasciare una certa percentuale di spazio libero in ogni blocco.

Il File ISAM viene diviso in due parti:

- **File Indice:** che contiene un record, composto di chiave-puntatore, per ogni blocco del file principale. In cui la chiave è la più piccola del suo blocco.
- **File Principale:** Il file realmente contenente i record

Ricerca

Per ricercare un record con chiave k **bisogna ricercare sul file indice una chiave k' che ricopre k** , cioè $tc\ k' \leq k$. Pertanto il costo della ricerca dipenderà dal costo di ricerca della chiave el file indice + il costo del trasferimento in lettura del blocco contenente il record cercato.

Poiché il file indice è ordinato sulle chiavi si può ricercare k' che ricopre k mediante **ricerca binaria**. Pertanto possiamo concludere che:

$$\text{COSTO_MAX} = \log_2(\text{BFI}) + 1$$

dove:

- $\log_2(\text{BFI})$ è il costo massimo della ricerca binaria nel caso peggiore (*poiché analizziamo tutti i Blocchi del File Indice*)
- +1 è il costo per caricare il blocco effettivo del file principale contenente il record cercato

Ricerca Per Interpolazione

La ricerca per interpolazione è basata sulla conoscenza della distribuzione dei valori della chiave. Ad Esempio, quando cerchiamo in un elenco telefonico non partiamo sempre da metà.

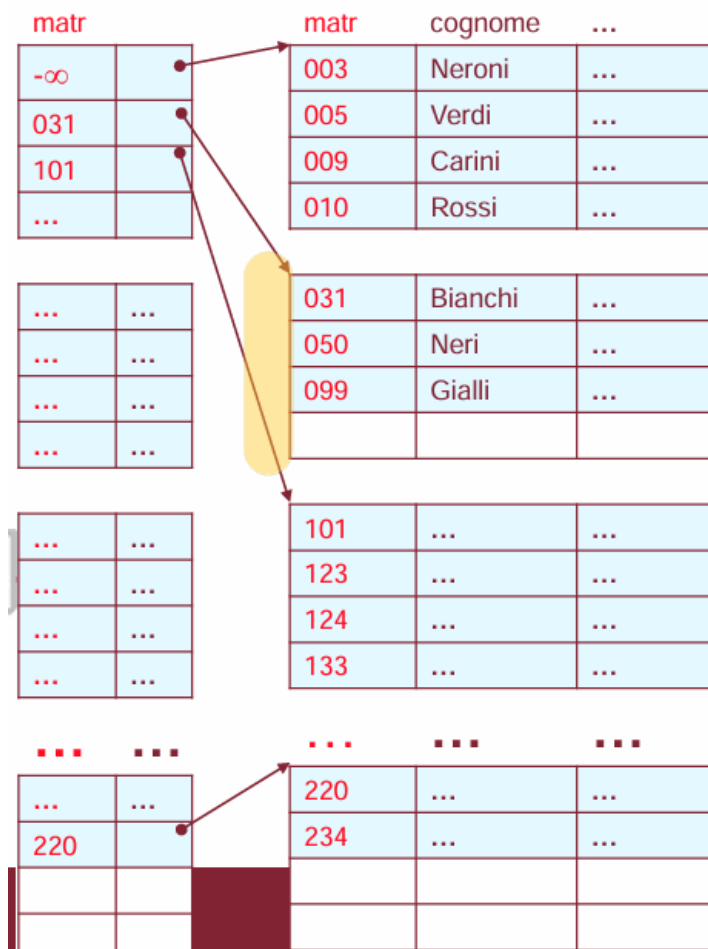
La ricerca per interpolazione richiede circa $\text{COSTO} = 1 + \log_2(\log_2(\text{BFI}))$ accessi, MOLTO più veloce ma ... MOLTO difficile conoscere f e poi la distribuzione dei dati potrebbe cambiare nel tempo.

Inserimento se c'è spazio

Dobbiamo effettuare ricerca per capire in quale blocco mettere il nuovo record. Se nel blocco c'è spazio semplicemente lo si inserisce gli altri slittano di 1.

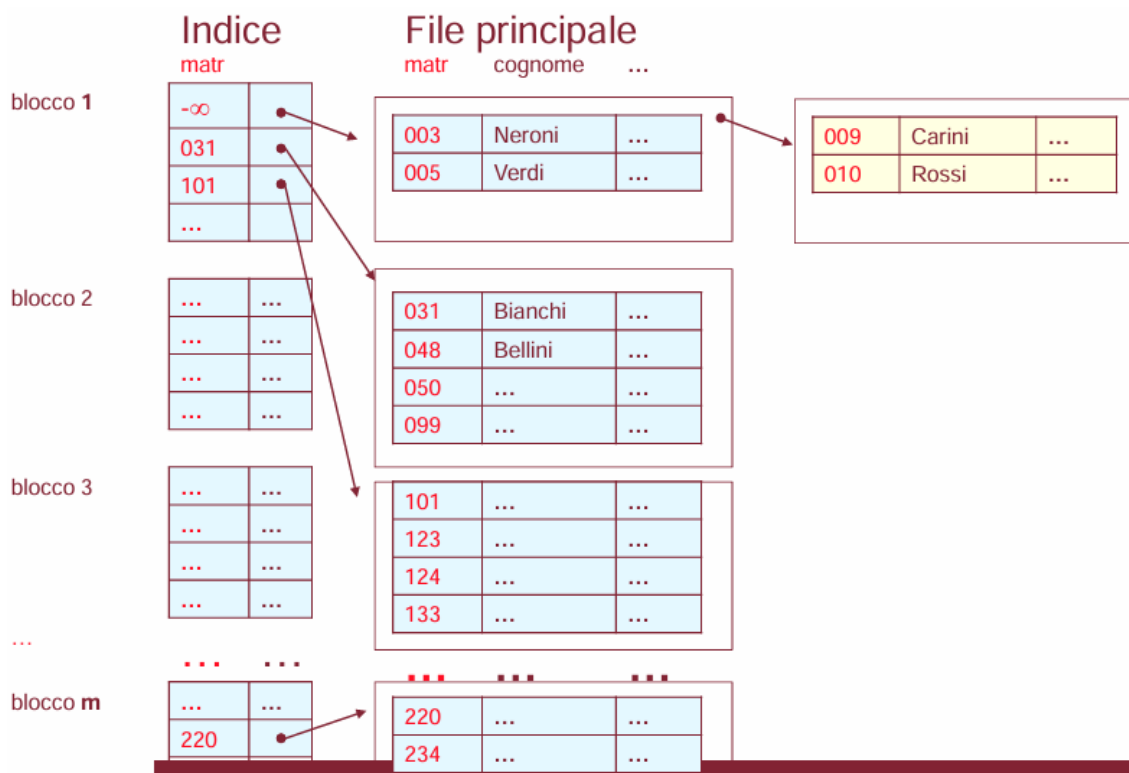
$$\text{COSTO} = \text{Ricerca} + 1$$

048	Bellini	
-----	---------	--



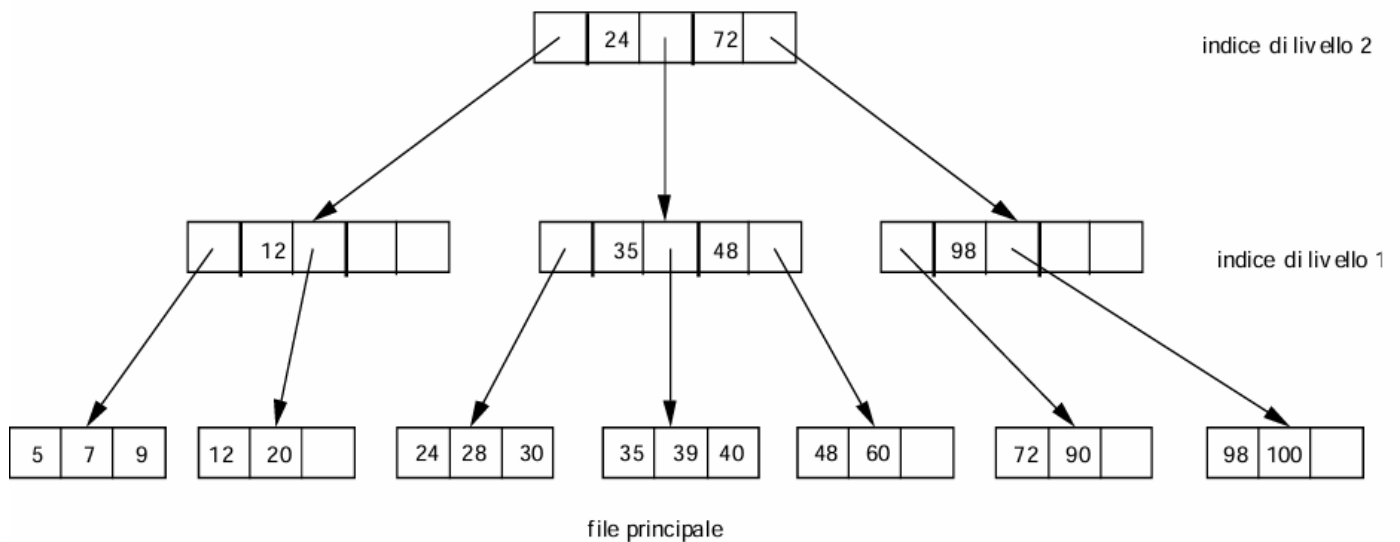
Tutte le altre operazioni non mi va di copiarle :)

File con Record Puntati



B-Tree

Il B-tree nasce dalla generalizzazione della struttura di indice. Si accede al file attraverso una gerarchia di indici.



La radice è costituito da un unico blocco e solitamente risiede sempre memoria principale durante l'utilizzo del file.

Per gli altri livelli invece, abbiamo un record indice, formato da chiave-puntatore, in cui ciascuna chiave **ricopre** quelle del suo sottoalbero.

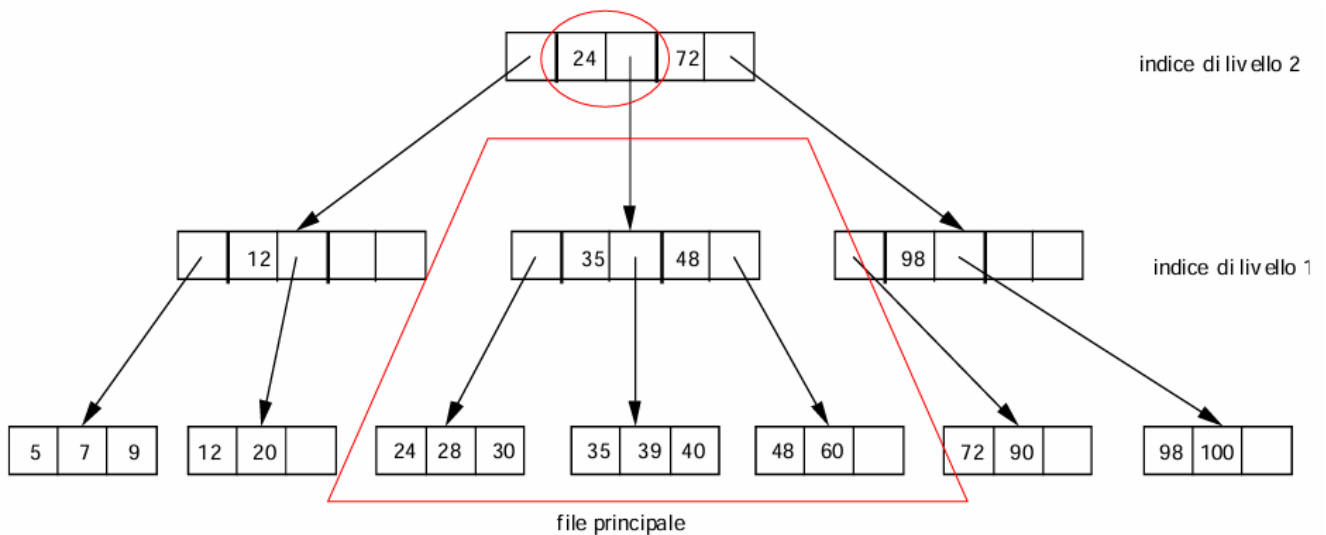
Ricerca

Come è facile immaginare, durante la ricerca di un record sarà fondamentale scorrere tutto l'albero, dalla radice fino al File Principale.

$$\text{COSTO} = h+1$$

Dove h è l'altezza dell'albero e + 1 è il costo per caricare il blocco effettivo per leggere il record cercato.

Esempio: nel seguente albero ricerchiamo il record con chiave 60.

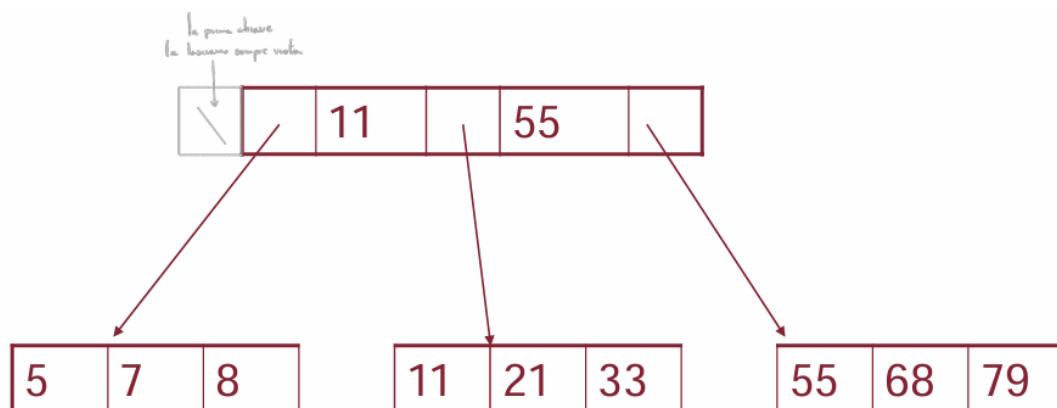


OSS: Più sono pieni i blocchi più h è piccolo (e quindi meno costa la ricerca). Conseguenza: si richiede che ogni blocco (sia del file principale che del file indice) sia pieno almeno per metà $\Rightarrow$ L'altezza massima dell'albero denotata con k si ha quando i blocchi sono pieni al minimo.

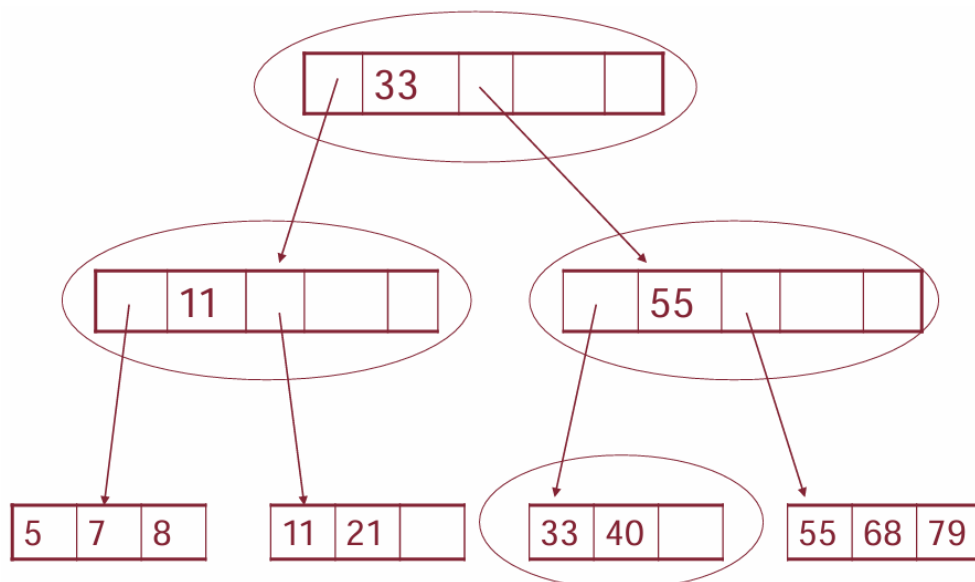
Inserimento con Blocchi Pieni

Se i blocchi sono completamente pieni, un inserimento può richiedere una modifica dell'indice ad ogni livello, comportando un aumento dell'altezza dell'albero (di un livello).

Esempio:



Si vuole inserire il record con chiave 40. Tuttavia ogni blocco deve essere pieno ALMENO al 50%, e dunque deve contenere almeno 2 record.



Inserimento se nei Blocchi c'è Spazio

Dobbiamo prima di tutto effettuare la ricerca, per capire in quale blocco andare ad inserire il record. Successivamente dobbiamo caricare il blocco (già compreso nel costo di ricerca) e riscriverlo in memoria secondaria (+1 accesso in scrittura)

$$\text{COSTO} = \text{Ricerca} + 1 = h+1+1 = h+2$$

Cancellazione

Simile all'inserimento. Bisogna prima ricercare